WeatherGuard Attendance

Joshua Clemens, Chima Ohaechesi, Roger Karam, Judah Fisher

Team04 · SE471SoftwareArchitecture · Spring2026

WeatherGuard Attendance — at a glance

A JavaFX app where students scan a session QR code and check in; the instructor's view shows status, weather, and end-of-session results.

Student phone

Scans the session QR code, posts check-in to Netlify.

→

MongoDB

Stores classes, students, sessions, attendance records.

→

Teacher View (JavaFX)

Shows roster, weather, QR code, end-of-session pie chart.

What we added - three design patterns wired into the Teacher View backend

Observer

TeacherViewController

Broadcasts each check-in as an immutable event.

State

SessionState (Inactive / Active / Closed)

Session lifecycle

Strategy

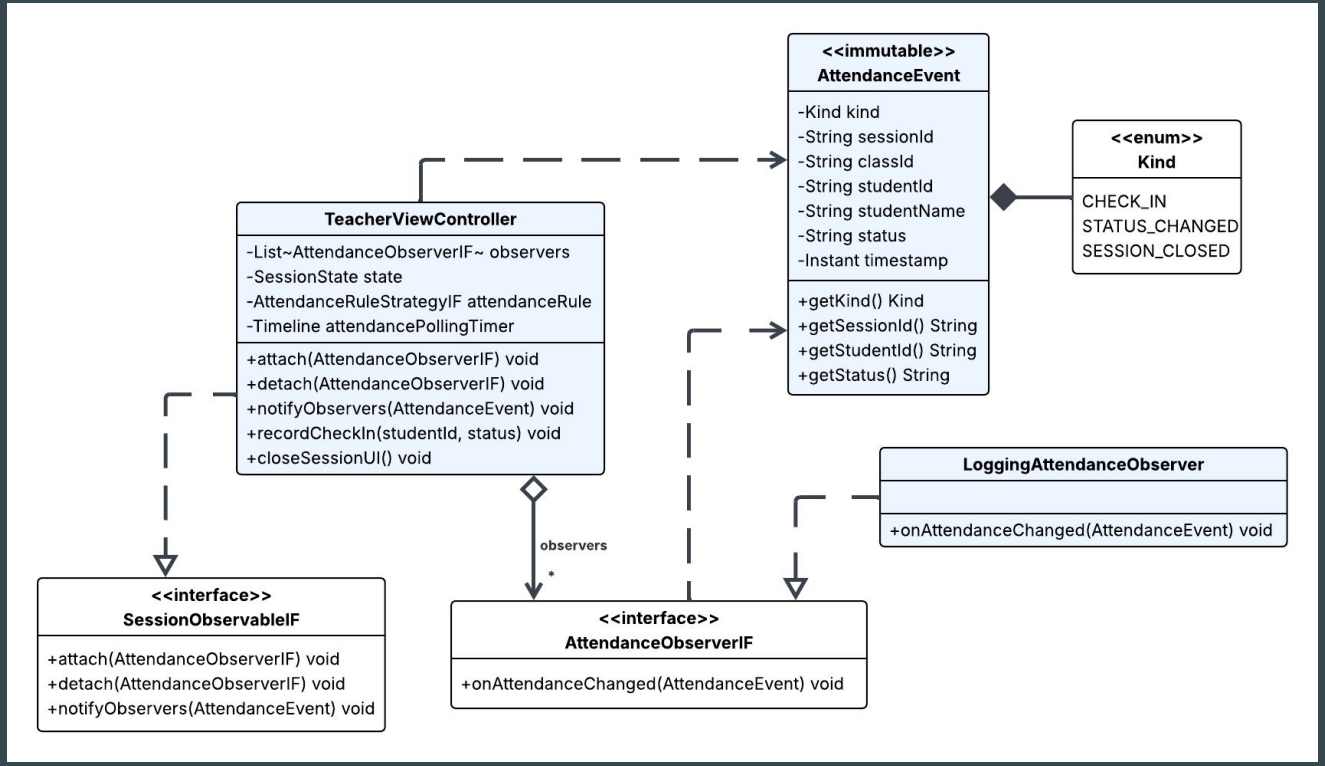
Attendance rule + Temperature unit

Pluggable rule + unit. Decorator wraps it.

Pattern catalog

Pattern	Where it's located	What it does
Facade	WeatherService	Hides the weather subsystem behind one API.
Strategy	TemperatureDisplayStrategyIF AttendanceRuleStrategyIF	Pluggable F/C unit. Pluggable present/late/absent rule.
Decorator	WeatherLeniencyStrategy	Wraps any attendance rule, adds grace minutes in extreme cold.
State	Inactive · Active · Closed	Session lifecycle as polymorphism.
Observer	SessionObservableIF LoggingAttendanceObserver	Broadcasts AttendanceEvent on every check-in.

Observer — class diagram



Why this pattern fits

Each check-in needs to fan out to the UI label, the pie-chart counter, and the console log — without those consumers knowing about each other.

TeacherViewController is the Observable. AttendanceEvent is the immutable payload. LoggingAttendanceObserver is the first concrete subscriber.

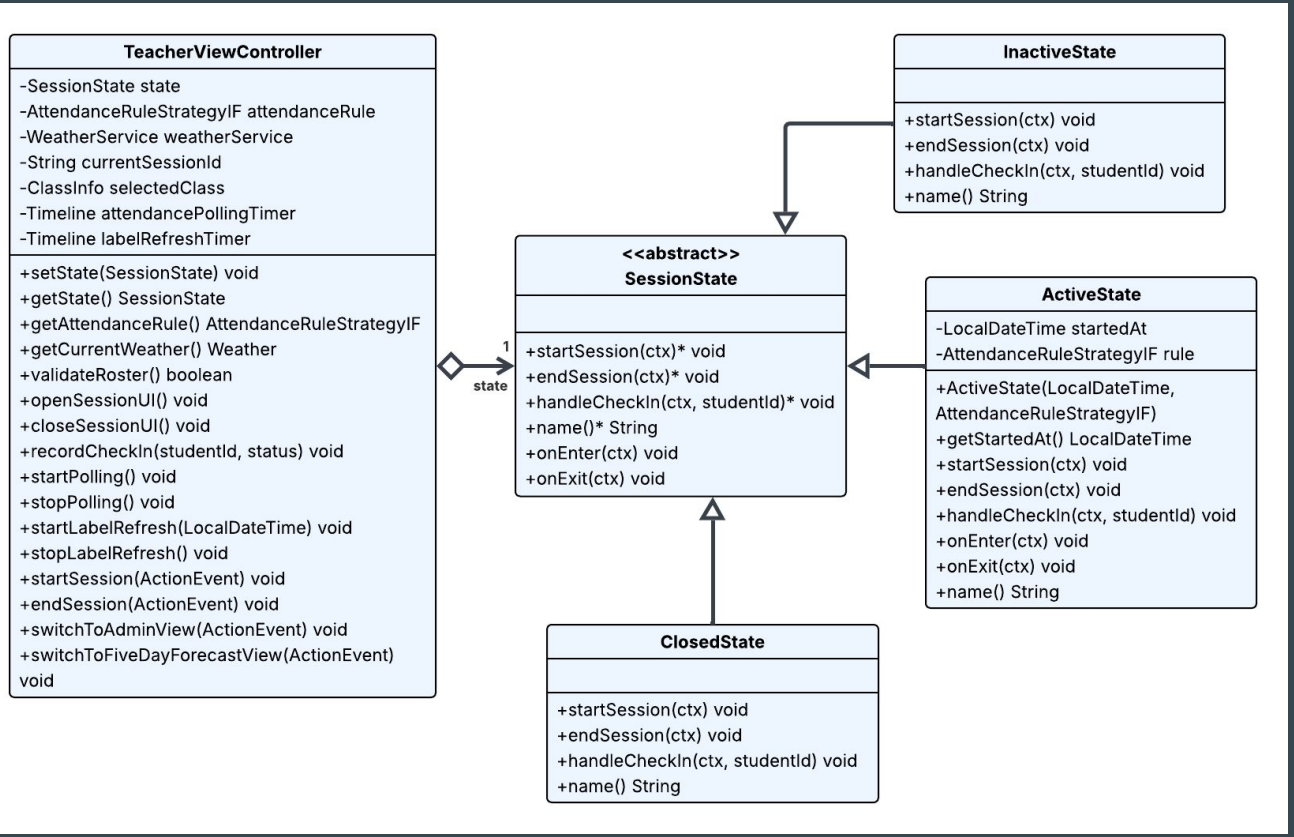
Adding analytics tomorrow = one new class, attached at startup. No controller changes.

Observer — in the code

```java
public void recordCheckIn(String id, String status) {
    studentStatusMap.put(id, status);
    AttendanceEvent e = new AttendanceEvent( Kind.CHECK_IN,
    currentSessionId, selectedClass.getClassId(), id, null,
    status, LocalDateTime.now());
    notifyObservers(e); // fan out to all subscribers
}
// initialize() – one line wires the first subscriber:
attach(new LoggingAttendanceObserver());
```

```
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S021 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S021 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S017 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S017 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S033 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S033 status=present
[AttendanceRule] WeatherLenient(+1min in cold, on Grace(1/10)): S027 -> present
[Observer] CHECK_IN sessionId=20260501_014241 classId=CS671 studentId=S027 status=present
```

State — class diagram



Why this pattern fits

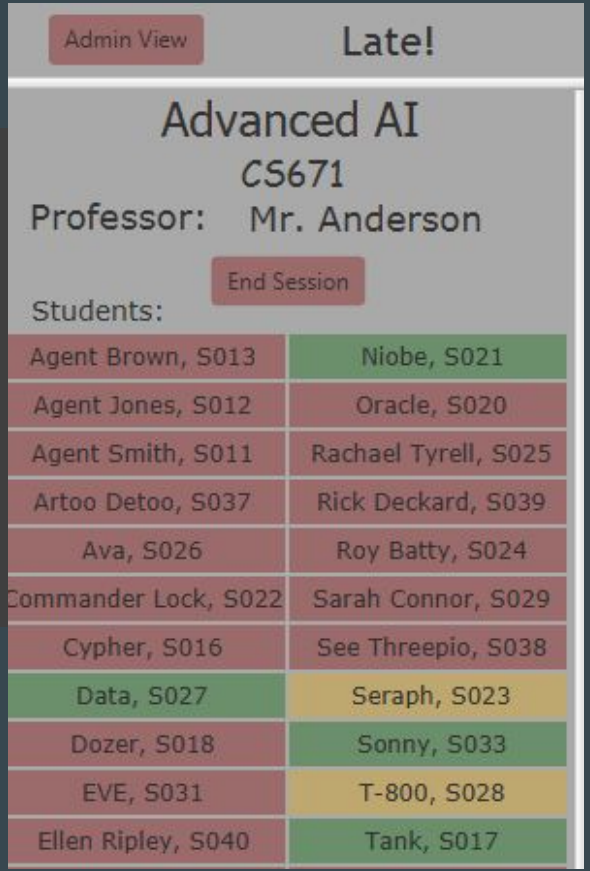
A session has three real lifecycle phases — not running, running, ended — and each phase wants different UI, different polling, different responses to a check-in.

Each state is a class. ActiveState carries its own startedAt and rule. Click handlers shrink to oneliners: state.startSession(this).

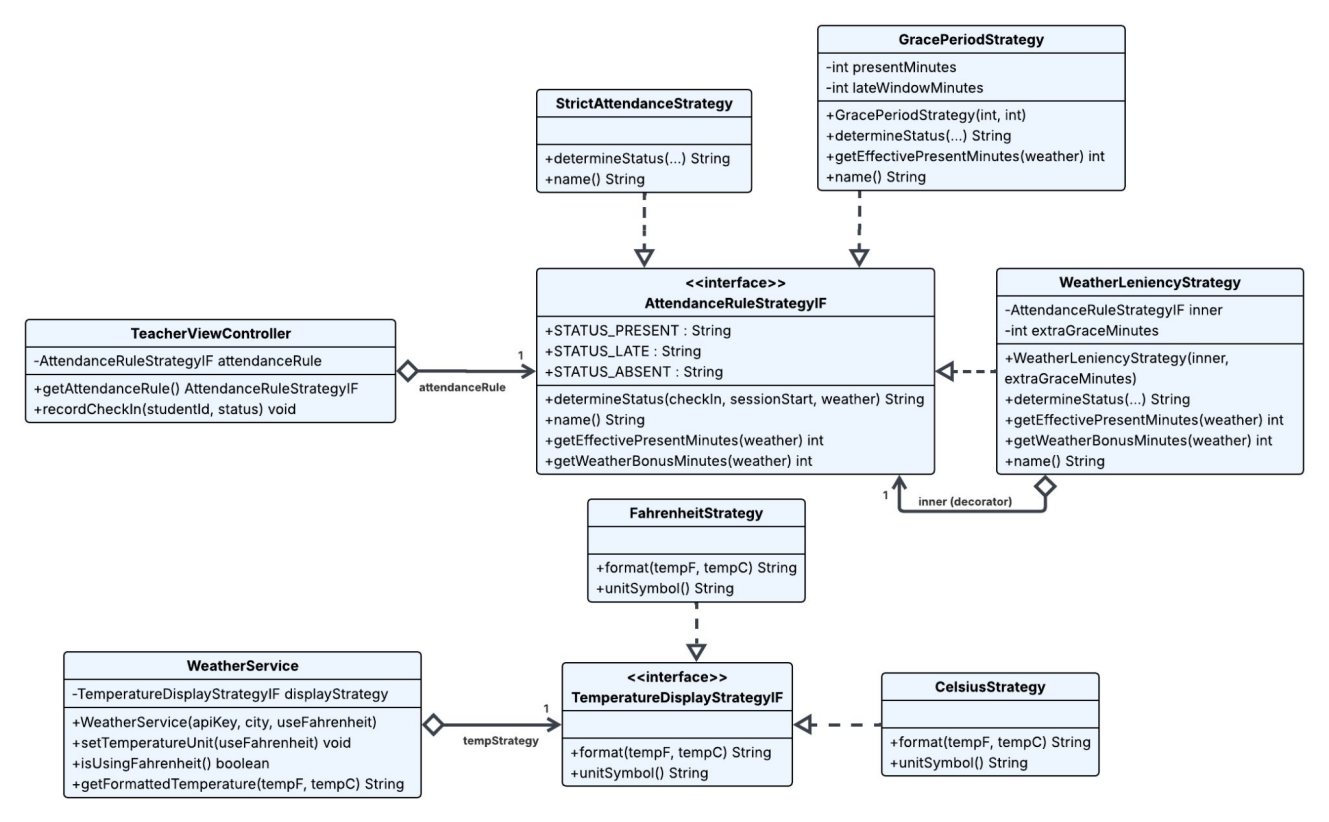
onEnter / onExit hooks make every transition atomic.

State — in the code

```
1   public void setState(SessionState s) {
2       state.onExit(this);
3       this.state = s;
4       state.onEnter(this);
5   }
6   // ActiveState.handleCheckIn — owns the rule
7   public void handleCheckIn(ctx, id) {
8       String status = rule.determineStatus(
9       LocalDateTime.now(), startedAt,
10      ctx.getCurrentWeather());
11      ctx.recordCheckIn(id, status);
12  }
```



Strategy — class diagram



Why this pattern fits

Two real choices in this app: which temperature unit to display, and which rule decides present / late / absent. Both should be swappable without touching controllers.

Strategy installed inside the Facade. New unit or new rule = one new class.

Bonus — Decorator WeatherLeniencyStrategy implements the same interface and wraps any rule, adding extra grace minutes in extreme cold.

Strategy — in the code

-30°C

-21°F

```
1   // WeatherService (the Facade) holds the unit:
2   private TemperatureDisplayStrategyIF tempStrategy;
3   public void setTemperatureUnit(boolean useF) {
4       tempStrategy = useF ? new FahrenheitStrategy(): new
        CelsiusStrategy();
5   }
6   // ActiveState carries the attendance rule:
7   private final AttendanceRuleStrategyIF rule;
8   // Decorator wraps any rule — same interface:
9   rule = new WeatherLeniencyStrategy(
10      new GracePeriodStrategy(1, 10));
```

Late in: 1:56

Time added: +1

How the patterns meet

One check-in. Five frames. Each pattern owns one frame and hands off through a typed argument.

POLL	TeacherViewController.checkForNewAttendance for each new attendance row → state.handleCheckIn(this, studentId)
STATE	ActiveState.handleCheckIn String status = rule.determineStatus(now, startedAt,weather);
STRATEGY	GracePeriodStrategy.determineStatus return checkIn ≤ presentCutoff ? PRESENT : LATE;
DECORATOR	WeatherLeniencyStrategy.determineStatus wraps the inner; may upgrade LATE → PRESENT in extreme cold
STATE	ActiveState.handleCheckIn ctx.recordCheckIn(studentId, status);
OBSERVER	TeacherViewController.recordCheckIn notifyObservers(new AttendanceEvent(CHECK_IN, ..., status));